

OOPJ Day 7

Tue, 15 Sept 26

final Keyword: Three Uses

- Applying final to any variable, method, or class makes it non-modifiable or non-extendable
- **Variables:** value cannot be changed once initialized
 - Local variable: `final int num = 5;` declared and initialized on the stack
 - Member variable: three valid initialization locations, only one may be used per variable:
 - At point of declaration: best when value is a known constant literal; JVM stores it in the constant pool
 - Constant pool allows memory sharing: two variables both holding 22 point to the same address; neither can modify it
 - Inside an initialization block: best when logic is needed (if/for/while/method call) to compute the value
 - Inside the constructor: best when the value is passed dynamically as a constructor parameter
 - These three locations are mutually exclusive: if initialized at declaration, cannot also initialize in block or constructor
- **Methods:** a final method in a parent class cannot be overridden in any derived class
 - e.g., Base has `show()` (overridable), `display()` (overridable), and `compute()` (final, cannot be overridden)
- **Classes:** a final class cannot be inherited by any other class
 - A new keyword for restricting inheritance more selectively (sealed) was introduced in Java 17 (covered below)

Method Overriding, @Override, and abstract

- All Java methods are virtual by default; no keyword is needed to mark a method as overridable
 - Unlike C++, which requires the virtual keyword explicitly
- **@Override annotation:** tells the compiler the intent is to override a parent method
 - Catches typos in method name, case changes, or signature mismatches that would otherwise silently create a new overloaded method instead of overriding
 - Written inside the derived class above the method declaration
- **abstract method:** declared with the abstract keyword, has no body
 - Forces every concrete derived class to provide an implementation
 - Syntax: `abstract int show(int x);`

- **abstract class:** a class containing one or more abstract methods must itself be declared abstract
 - Cannot be instantiated directly; object creation only possible through a concrete derived class
 - Indirectly instantiated: creating a Man object first creates a Human (abstract) object inside it
 - Can contain a mix of concrete methods, abstract methods, and final methods
 - Final methods inside an abstract class cannot be overridden; abstract and concrete methods can be
- **Contradiction rule:** a class cannot be both abstract and final
 - abstract requires inheritance to be useful; final forbids inheritance entirely

Java Naming Conventions

- Naming conventions are mandatory in Java; violations in frameworks like Spring, Hibernate, or Servlets can cause cryptic runtime exceptions (e.g., capitalizing one letter in a method name can produce a 5–6 page stack trace)
- **Classes, interfaces, annotations, enums:** PascalCase — first letter of every word capitalized
 - e.g., SalesPerson, Manager, SpecialSalesManager, SalesManager
- **Methods and variables (both local and member):** camelCase — first letter lowercase, first letter of every subsequent word capitalized
 - Also called camel case: “camel’s head is low, then it rises”
 - e.g., basicSalary, showData, amountTransactionDetails
- **Constants (final variables):** ALL_CAPS with underscores
 - e.g., final int MAX = 8;, final int ADD_MANAGER = 1;
- **Packages:** all lowercase, typically a single word per segment, structured as reverse domain name
 - e.g., in.cdac.employee, org.pankaj.utils
 - cdac.in domain becomes in.cdac as a package prefix
- **Code review feedback on Shivam’s Employee project:**
 - Class named CommentDetails: inappropriate, unclear purpose — avoid vague class names
 - Variable named c: unreadable; no one can tell what c represents without context
 - Magic numbers like 1, 2, 3 in if/switch conditions: replace with named final constants
 - e.g., final int ADD_MANAGER = 1; makes if (choice == ADD_MANAGER) self-documenting
 - If the value changes later, update only the constant declaration; all usages update automatically
 - Inconsistent naming: some methods followed conventions, others did not — consistency is required throughout
 - Using System.out.println or console input anywhere outside the main function is wrong

- Employee system could be used in console, GUI, mobile, or web applications
 - Embedding I/O inside display() or other methods makes code console-specific and non-reusable
- Separate Menu class: unnecessary — a standalone function for the menu is sufficient; no need for a dedicated class
- protected keyword in C++ applies to a block; in Java, every declaration requires its own access specifier individually

Switch Expressions (Java 14 / Java 17)

- **Classic switch (pre-Java 14):**
 - Supports int only up to Java 4; String added from Java 5
 - Fall-through: if a case has no break, execution continues into the next case until a break is hit
 - e.g., entering 1 with no break after case 1 prints both case 1 and case 2
- **Switch expressions introduced in Java 14:**
 - Use the arrow operator -> instead of the colon :
 - No fall-through: each case is automatically terminated; no break statement needed or permitted
 - Multiple statements per case must be wrapped in {}
 - Syntax example: switch (choice) { case "add" -> System.out.println("Case 1"); case "delete" -> System.out.println("Case 2"); default -> System.out.println("Default"); }
- **Yielding a value (enhanced further in Java 17):**
 - Switch expressions can return (yield) a value, unlike classic switch statements
 - The result is assigned to a variable: int result = switch (choice) { ... };
 - Single-expression cases: value is returned directly without yield
 - Multi-statement cases: use the yield keyword inside {} to return the value
int result = switch (choice) { case 1 -> 11; default -> { System.out.println("Default"); yield 12; } };
 - Practical use: determine what a user selected (add manager, add engineer, display, etc.) and capture the result for downstream logic
- **Summary note to write down:**
 - Java 14 enhanced switch by preventing fall-through with the introduction of switch expressions
 - Cases in a switch expression do not require a break; the next case acts as automatic termination
 - Switch expressions were further enhanced to return a result value using the yield keyword inside curly braces

Sealed and Non-Sealed Classes (Java 17)

- **Problem before Java 17:** only two options existed for a class

- Open: any class anywhere could inherit it
- Final: no class could inherit it at all
- No middle ground for controlled, selective inheritance
- **sealed class:** restricts which classes are permitted to inherit it
 - Syntax: `public sealed class Base permits Derived, SpecialDerived {}`
 - Only the classes listed in the permits clause can extend Base; all others get a compile error
 - This is called conditional inheritance
- **Each permitted subclass must declare one of three modifiers:**
 - final: cannot be further inherited; inheritance chain ends here
 - sealed: must itself declare its own permits list; inheritance continues in a controlled way
 - non-sealed: reopens inheritance publicly; any class can now extend this subclass
- **Demonstration from class:**
 - Base is sealed, permits only Derived
 - Derived declared as non-sealed: `DerivedMost` can extend `Derived` freely
 - `SpecialDerived` not listed in Base's permits: compile error regardless of its own modifier
- **Summary note to write down:**
 - Prior to Java 17, Java classes used open or public inheritance: any class could inherit a non-final parent
 - Java 17 introduced sealed and non-sealed keywords
 - A sealed class must provide a permits list; only listed classes can inherit it
 - Each permitted subclass must itself be declared final, sealed (with its own permits), or non-sealed (open inheritance)

Packages and Access Specifiers

- **Why packages exist:** resolve class name ambiguity when two libraries define a class with the same name
 - e.g., both Oracle and Microsoft ship a `Date` class; importing both causes a compile-time ambiguity
 - Solution: qualify the class with its package: `org.pankaj.Date` vs `in.cdac.Date`
 - Fully qualified class name can be used inline without an import statement to resolve ambiguity explicitly
- **Package declaration rules:**
 - Must be the first statement in any Java file; nothing can precede it
 - Directory structure on disk must exactly match the package name
 - Mismatch between declared package and folder location causes a compile error: "declared package does not match expected package"
- **Import statements:**
 - Required to use a public class from a different package without writing its fully qualified name every time
 - `import org.pankaj.Date;` — imports one specific class

- `import org.pankaj.*;` — wildcard imports all public classes in the package
- **Never use wildcards in practice:** zero performance difference (Java uses dynamic linking, unlike C++ which embeds all classes into the executable), but wildcards hide which classes the file actually depends on
 - When shipping code to a client, dependencies must be identifiable from the import list
 - Use Ctrl+Shift+M in Eclipse to import the class under the cursor; choose from a list if multiple packages offer the same name
 - Use Ctrl+Shift+O to organize imports: auto-adds missing ones, removes unused ones
 - Ctrl+Shift+L lists all Eclipse shortcuts
- **Default (no) package:** strongly discouraged; classes in the default package cannot be imported by any other package
- **java.lang package:** auto-imported by the compiler into every Java file; contains String, Integer, Boolean, Double, Float, Short, Long, Character, @Override, and all other core language classes — no explicit import needed
- **Access specifier matrix:**

Specifier	Same package	Different package, no inheritance	Different package, inherited	default
Yes	No	No private	No	No
	No	No protected	Yes	No
			Yes	public
			Yes	Yes
			Yes	Yes

- **Public class rule:** a public class must reside in a Java file with exactly the same name as the class; required for incremental compilation and JVM class lookup
- **Package-scope (default) class:** usable only within its own package; attempting to import it from outside gives a “not visible” error, not a “not found” error
- **Practical instruction:** move all existing classes into a named package (e.g., `in.cdac`) by right-clicking `src`, creating a new package, and dragging classes in; Eclipse adds the package declaration automatically

Class Loading and Static Initialization Order

- The JVM is not a single entity; it comprises multiple modules, each running as a thread:
 1. ClassLoader, Garbage Collector, Memory Manager, Thread Scheduler, Security Manager, Exception Handler, Constant and String Pool Manager
 2. All modules load automatically when any Java application launches
- **ClassLoader’s job:** load `.class` bytecode files into memory dynamically, on demand
- **Three conditions that trigger class loading** (whichever occurs first):
 1. First time an object of the class is created (new)
 2. First time a static variable or static method of the class is accessed
 3. Programmer explicitly loads the class: `Class.forName("in.cdac.Base")`
- **Important:** `Base objBase = null;` does NOT trigger class loading
 1. Only a reference is created; no object, no static access, no explicit load — none of the three conditions are met
- **Sequence of execution when a class is loaded:**

1. Static data members allocated and initialized
 2. Static initialization block executed (once only, never again for the lifetime of the class)
 3. (If triggered by object creation) Instance variables allocated and initialized
 4. Instance initialization block executed
 5. Constructor executed
- **Demonstration:** accessing a static variable before creating any object fires only steps 1–2; steps 3–5 occur only when new is used

Upcasting, Downcasting, and RTTI

- **Core problem:** storing Manager, Engineer, and Salesperson objects in three separate arrays is redundant; all three are Employees
 - Better design: single Employee[] arrEmployee = new Employee[50];
 - Upcasting allows one array to hold objects of all three derived types
- **Upcasting:** assigning a derived object to a base-type reference
 - arrEmployee[count] = new Manager(name, address, age, gender, salary, hra);
 - The reference type is Employee; the actual object is Manager — this is implicit upcasting
 - Through this reference, only Employee members are visible; Manager-specific members (e.g., HRA) are hidden from the compiler
- **Downcasting:** explicitly casting a base reference back to its derived type to access type-specific members
 - Manager objManager = (Manager) arrEmployee[i];
 - Then objManager.getHRA() becomes accessible
 - Performing a downcast to the wrong type (e.g., casting an Engineer object to Manager) throws a ClassCastException at runtime
- **RTTI — Runtime Type Identification:** determines the actual type of an object at runtime
 - Java keyword: instanceof operator
 - Internally checks the virtual table pointer of the object to confirm its actual type
 - Every Java object carries a VTable pointer; Manager objects point to Manager's VTable, Engineer objects to Engineer's VTable
 - instanceof compares that pointer against the target type
- **Safe downcast pattern:**

```
if (arrEmployee[i] instanceof Manager) { Manager m =
(Manager) arrEmployee[i]; System.out.println(m.getHRA()); } else if (arrEmployee[i]
instanceof Engineer) { Engineer e = (Engineer) arrEmployee[i];
System.out.println(e.getOvertime()); }
```
- **Generalization principle:** the higher up the hierarchy a reference points, the more generalized the code; come down the hierarchy only when type-specific member access is required

Shallow Copy, Deep Copy, and Java Strings

- **Shallow copy (member-wise copy):**
 - When an object is copied, all field values are copied as-is
 - For primitive fields: independent copies — changing one does not affect the other
 - For pointer/reference fields: both objects end up pointing to the same underlying memory address
 1. Modifying the shared memory through one object affects the other
 - This is the default behavior of the copy constructor in C++ and of Java's string pool
- **Deep copy:**
 - Allocates new memory for each referenced field and copies the contents
 - Changes to one object's referenced data do not affect the other
 - Implemented by overriding the copy constructor (C++) or manually allocating and copying in Java
- **Java String pool:**
 - Strings are objects in Java (not primitives); dot operator works on string literals directly
 - Each character in Java is 2 bytes (Unicode support), making strings memory-heavy
 - To avoid redundant allocation, JVM maintains a string pool (part of the constant pool)
 - First use of a string literal creates an object and registers it in the pool
 - Subsequent uses of the same literal reuse the same object (shallow copy behavior)
 - String str = "hello"; String str2 = "hello"; — both point to the same pool object (same ID in debugger)
 - String str3 = "Hello"; — different case, different object
 - String str4 = new String("hello"); — bypasses the pool; always creates a new object
- **String immutability:**
 - Once a String object is created, its value cannot be changed
 - str = "bye"; does not modify the "hello" object; it creates a new "bye" object and redirects the reference
 - This prevents the shallow-copy problem: str1 still points to "hello" even after str is reassigned to "bye"
 - Immutability is what makes the string pool safe to use
- **Comparing strings:**
 - == compares memory addresses (reference equality): true only if both references point to the same object
 - .equals() compares string contents regardless of memory location
 - .equalsIgnoreCase() compares contents ignoring case differences
- **Concatenation and object creation cost:**
 - String str = "hello" + "bye" + "hello" + "bye"; creates 5 objects:
 1. "hello" (pool)
 2. "bye" (pool)

3. "helloBye" (intermediate)
4. "helloByehello" (intermediate)
5. "helloByehellobye" (final, assigned to str)
 - Objects 3 and 4 are immediately eligible for garbage collection
 - Repeated concatenation wastes memory and degrades performance
- **StringBuffer — mutable alternative:**
 - Use StringBuffer when building strings through repeated concatenation
 - Single mutable object; no intermediate objects created
- `StringBuffer buffer = new StringBuffer("hello"); buffer.append("bye");
buffer.append("hello"); buffer.append("bye"); String str = buffer.toString();`
 - Only one object exists throughout; final string extracted with `.toString()`

Administrative Announcements and Assignment

- **Mid-quiz:** moved to tomorrow; will be held after the AWS and Linux session (approx. 4:30–5:30 PM)
- **CR elections:** also tomorrow after the quiz
 - Each candidate gets 60–90 seconds to address the class before voting opens
 - Voting via a Google Form (no NOTA option; one selection required; candidates may also vote)
 - Two categories: Technical CR and Communication CR
 - Tie-breaker: faculty panel interview with the tied candidates
 - Meeting link end date was 14th September; being updated to 24th September
- **Revision sessions:** faculty requested topic/doubt submissions from students; none received yet
 - Revision sessions will begin the day after tomorrow once tomorrow's session is complete
- **Assignment due tomorrow (Shivam and all students):**
 - Refactor Employee project to apply all naming conventions
 - Move all classes into a proper named package (e.g., `in.cdac`)
 - Replace magic numbers with named final int constants (`ADD_MANAGER`, `ADD_ENGINEER`, etc.)
 - Use a single `Employee[]` array with upcasting instead of separate arrays per type
 - Guard all downcasts with `instanceof` (RTTI)
 - Implement four operations: add employee, display all employees, sort by designation, delete employee
 - Remove all `System.out.println` and console input from any method other than `main`

Chat with meeting transcript:

<https://notes.granola.ai/t/0338910f-9236-4905-bcbf-41edd48b6f6f-00demib2>